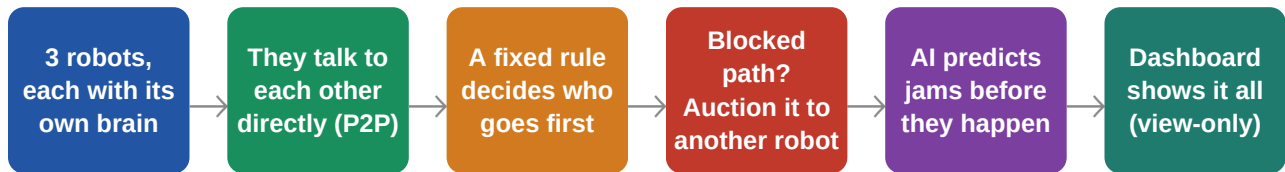


The Full Reasoning Chain

From a Centralized Robot Fleet to a Working Decentralized 3-AMR System

What We're Building — In One Glance



Then we prove it: run the same task with and without this system, and show ours is faster with zero crashes.

Part 1 — Remove the Boss Computer

1

Fleet grows, central computer struggles

More robots join. The central “boss” slows down and becomes risky. So we remove it.

TECH → Design decision — no code yet.

FIXES → Central server = single point of failure

2

Give each robot its own brain

Each robot gets its own onboard computer (Pi/Jetson), simulated as 3 separate programs on one laptop.

TECH → ROS2 namespaces: /amr_1, /amr_2, /amr_3

FIXES → Must run on edge hardware, not cloud

3

Each AMR is a separate agent

Each robot simulates having its own independent brain.

TECH → ROS2 node per robot + Gazebo spawns 3 robot models

4

Robots still need to coordinate

Connect them over a shared local channel that mimics a real wireless mesh.

TECH → Zenoh (rmw_zenoh) — true peer-to-peer, no server

FIXES → Network lag from querying one central point

Part 2 — Talk & Avoid Collisions

5

Robots broadcast their status

Each AMR shares its position and planned path. Every other AMR can hear it. Careful: if each robot only publishes inside its own namespace, others may not actually receive it — coordination would silently fail with no error shown.

TECH → [Global channel names \(e.g. swarm/amr_1/telemetry\), every robot subscribes to swarm/*/telemetry](#)

6

Warehouse has tight aisles on purpose

A shared map with narrow choke points, so robots are sometimes forced into conflict.

TECH → [Gazebo warehouse world design](#)

7

Two robots want the same spot

Their paths cross at the same place, same time. A conflict.

TECH → [Space-Time check: robot positions tracked as \(x, y, time\)](#)

FIXES → [Must detect collisions in real time](#)

8

A fixed rule decides who goes first

100% reliable math — not a guess. Longer wait or lower battery wins right-of-way. Never AI here: a wrong guess means a crash.

TECH → [Priority Aging formula, written in Python](#)

FIXES → [Safe, provable right-of-way decisions](#)

9

Make sure the rule is fair

Check it doesn't always make the same robot lose.

TECH → [Wait-time keeps increasing until it wins — no one loses forever](#)

FIXES → [Circular deadlock \(A waits B, B waits C, C waits A\)](#)

9b

Fix the “after-you” freeze

Two equal-priority robots keep yielding to each other, forever — like a hallway dance.

TECH → [Random 10–200ms delay breaks the tie](#)

FIXES → [Livelock](#)

Part 3 — Handle Blocked Paths

10

Detect a blocked aisle for real

UPDATED

One robot's path gets obstructed. Instead of just pretending it's blocked, the robot's own camera sees and identifies what's actually in the way — then decides what to do based on what it is.

OLD → Manual test trigger in simulation

NEW → Onboard RGB camera + YOLOv8n-nano (ONNX) classifies the obstruction: box/pallet → triggers auction, human worker → immediate motor cutoff + alert, another robot → starts negotiation instead

FIXES → Aisle blockages were only simulated, never actually detected — also adds a real safety layer for humans

11

Others re-plan instantly

No permission needed — they recalculate their own paths live.

TECH → Nav2 replans locally on each robot

FIXES → Adapt locally, no cloud dependency

12

The stuck robot's job goes up for grabs

Other robots check distance and availability. Best one takes over automatically.

TECH → Contract Net Protocol (CNP) auction

FIXES → Permanently blocked path needs handoff

12
b

Handle the messy real cases

Everyone busy? Auction message lost? Two robots bid at once? False alarm? A robot freezes mid-move waiting for the auction to finish? Small rules handle each case cleanly.

TECH → Bid-while-busy check, auto-retry, tie-breaker, second-sensor confirm, and a clear auction sequence (waiting → assigned) with a 500ms timeout so a robot never waits forever

FIXES → CNP edge cases, robots freezing mid-auction

13

All 3 robots broadcast their data

Position, battery, task — build a dashboard that just listens.

TECH → Dashboard subscribes to the same broadcast channel

Part 4 — Survive Real-World Mess

14

Test what happens when Wi-Fi is bad

Real warehouses have metal shelves that block signal. We copy this on purpose: run each robot in its own separate box (container) on the laptop, and make some of its messages arrive late or get lost completely, just like a real dead-zone would. One thing to watch: all 3 robots must still share the same simulation clock, or the timing checks in Step 7 go wrong and cause false alarms.

TECH → Docker containers to isolate each robot, tc/netem to fake packet loss, use_sim_time: true to keep clocks in sync

FIXES → Wi-Fi dead-zones cause aisle-blocking jams; clock drift causes false collision warnings

15

Robot doesn't panic when it can't hear others

Falls back to its own camera/LiDAR. Slows down instead of freezing.

TECH → LiDAR-only navigation, speed capped ~40%

FIXES → No single lost robot should jam the fleet

16

What if a robot goes fully silent?

Wait a bit first — maybe just bad signal. If it stays silent too long, assume it's dead and reassign its task.

TECH → 2-tier timeout: ~2s = wait it out, ~6s = reassign

FIXES → Tell a dropout apart from an actual failure

17

What if a robot carries something heavy?

A loaded robot can't stop as fast. Give it more space and time.

TECH → Bigger safety buffer when carrying a payload

FIXES → Loaded vs empty robots brake differently

17
b

What about bad data from a glitching robot?

UPDATED

Don't blindly trust broadcasts. Use the same onboard camera to visually confirm a neighbor is actually where it claims to be, instead of only comparing numbers.

~~OLD → Compare claimed position to your own sensor reading (numbers only)~~

NEW → Reuse the YOLOv8n vision model from Step 10 — if a robot is visible in-frame, check its detected position against its broadcasted position; mismatch = flag and ignore its data temporarily

FIXES → A malfunctioning robot could mislead the fleet — now caught visually, not just numerically

17
c

What about battery running out?

Don't take a job you can't finish. Head to charge on your own once battery is low.

TECH → Battery check (~25%) before bidding on tasks

FIXES → Wasted fleet capacity from poor charge planning

Part 5 — Dashboard, AI & Final Proof

18

Build the live dashboard

Shows all 3 robots' positions, battery, tasks in real time. This is your visual proof for judges.

TECH → Web dashboard (Flask + HTML/JS) or RViz2

19

Dashboard only watches, never commands

It never sends anything back to the robots — proves the system is truly decentralized.

TECH → Dashboard subscribes only, never publishes

19
b

Predict congestion before it happens

Predict which aisle gets crowded in the next few seconds. Reroute before it happens, not after.

TECH → Small trained model (XGBoost / small MLP)

FIXES → System was reactive only — this makes it predictive

20

Build and run the full system

Run the same scenario as a “dumb” stop-and-wait version too, and time it.

TECH → Automated Python test script for both versions

21

Prove it with numbers

Your version should be faster with zero collisions. Run it many times, not once, so it's a real result.

TECH → 20 repeated runs, compare average results

FIXES → A single lucky run isn't real proof